

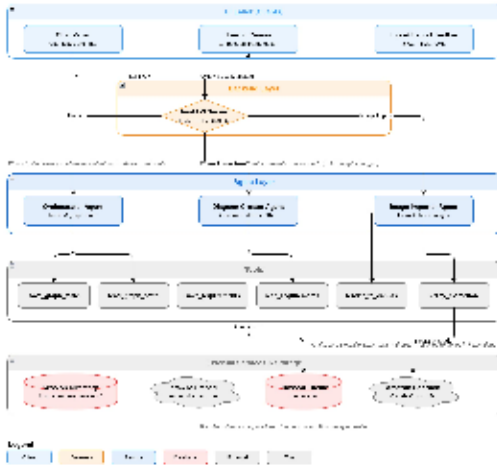
Agentic_Prez/ArchVision

Scope

ArchVision AI is an agentic AI-powered architecture diagramming platform that generates editable [draw.io](#) diagrams from natural language descriptions or existing diagram images. It covers text-to-diagram generation via chat, diagram modification through conversational commands, AI-powered image import with majority-voting verification to reduce hallucinations, and full [draw.io](#) canvas editing. The platform supports export in JPG, HTML (interactive with [draw.io](#) viewer), and XML ([draw.io](#) native) formats, and import from JPG/PNG images, HTML, and XML files. All visual output uses Ericsson branding — brand color palette with pastel fills, Ericsson Hilda font, and the Ericsson logo.

Architecture

Architecture Diagram:



- **Frontend — React + TypeScript**
 - Single-page application with a chat panel and embedded [draw.io](#) canvas
 - Communicates with backend via REST API with real-time status polling for loading indicators
 - Key views: Chat Panel, [draw.io](#) Canvas (iframe), Import/Export toolbar
 - Two modes: Standalone (normal diagramming) and Presales Integration (auto-generates from URL context, "Return to Bot" sends image back)
- **Backend — FastAPI (Python)**
 - Central orchestrator coordinating a 2-phase agent pipeline
 - Exposes REST endpoints for chat, image import, status polling, callback proxy, and session reset
 - Manages session state in temporary directories — graph state JSON and requirements markdown per session
 - Colors dynamically computed from Ericsson theme.json (single source of truth) and injected into agent prompts at runtime
- **AI Agent Layer — 3 LangGraph ReAct Agents**
 - Orchestrator Agent — understands user intent (create, modify, or question), gathers requirements, builds and updates graph state JSON representing diagram components and connections
 - Diagram Creator Agent — reads graph state and requirements, generates complete [draw.io](#) XML with Ericsson brand styling (pastel fills, brand stroke colors, proper shape types), supports both full generation and incremental modification of existing diagrams
 - Image Importer Agent — extracts components from uploaded diagram images, runs majority-voting verification (3 parallel independent LLM re-reads with fuzzy cross-referencing to remove hallucinated components), generates [draw.io](#) XML from verified extraction
 - All agents use Amazon Bedrock (Claude Opus) as the LLM provider
- **Tools — 6 Agent Tools**
 - save_graph_state — writes graph JSON (nodes, edges, annotations) to session directory
 - read_graph_state — reads current graph JSON from session
 - render_to_canvas — stores generated [draw.io](#) XML as pending output for frontend
 - save_requirements — writes architecture requirements markdown to session
 - read_requirements — reads requirements from session
 - verify_extraction — performs 3 parallel independent LLM image reads, cross-references against extraction using fuzzy word matching, removes unconfirmed components
- **Canvas — [draw.io](#) Embedded**
 - [draw.io](#) editor embedded via iframe ([embed.diagrams.net](#))
 - Communication via postMessage — load XML, export XML, configure branding
 - Ericsson brand colors and font registered via [draw.io](#)'s configure API
 - Full editing capabilities — drag, resize, connect, add shapes, undo/redo
- **Storage**
 - Session directories (/tmp/agentprez_sessions/) — graph_state.json, [app_requirements.md](#) per session
 - In-memory state — current canvas XML, pending XML, processing phase
 - No persistent database — sessions are ephemeral

- **External Dependencies**
 - Amazon Bedrock — LLM provider (Claude Opus) for all three agents
 - embed.diagrams.net — draw.io editor iframe (requires internet)
 - viewer.diagrams.net — draw.io viewer for HTML export and JPG rendering

UC's

- **UC1 — Create a diagram from natural language**
A user describes an architecture in the chat panel (e.g., "Create a microservices architecture for a food delivery app"). The Orchestrator Agent gathers requirements and builds a graph state. The Diagram Creator Agent generates draw.io XML with Ericsson branding and renders it on the canvas as a fully editable diagram.
- **UC2 — Modify an existing diagram via chat**
The user requests changes to the current diagram (e.g., "Add a Redis cache", "Remove the monitoring service", "Connect Order Service to Payment Gateway"). The Orchestrator Agent updates the graph state with only the requested changes. The Diagram Creator Agent modifies the existing canvas XML — preserving all positions and styles — rather than regenerating from scratch.
- **UC3 — Ask questions about the diagram**
The user asks about the current diagram (e.g., "What components are there?", "How many services?"). The Orchestrator Agent reads the graph state and responds with a text answer. No diagram changes are made.
- **UC4 — Import an architecture diagram image**
The user uploads a JPG/PNG screenshot of an existing architecture diagram. The Image Importer Agent extracts all visible components and connections, runs majority-voting verification (3 parallel LLM reads) to remove hallucinated components, and generates an editable draw.io diagram on the canvas.
- **UC5 — Import an existing draw.io XML file**
The user imports a .drawio.xml file. The diagram loads directly into the canvas and syncs with the backend so the agent can answer questions and make modifications.
- **UC6 — Import a draw.io HTML file**
The user imports an HTML file containing an embedded draw.io diagram. The system extracts the XML (from base64-encoded data, script tags, or raw mxGraphModel tags) and loads it into the canvas.
- **UC7 — Export diagram as JPG**
The system renders the diagram in a hidden iframe using the draw.io viewer, captures the SVG with tight bounding box (no whitespace), converts to JPEG via canvas at 2x resolution, and downloads the file.
- **UC8 — Export diagram as interactive HTML**
The system embeds the draw.io XML (base64-encoded) in an HTML file with the draw.io viewer script. The exported file opens in any browser as an interactive, zoomable diagram.
- **UC9 — Export diagram as XML**
The system exports the raw draw.io XML as a .drawio.xml file, directly editable in diagrams.net or any draw.io-compatible tool.
- **UC10 — Manual editing on canvas**
The user directly edits the diagram on the draw.io canvas — dragging shapes, resizing, connecting, adding new shapes, changing colors. All manual changes are preserved in exports and reflected when the agent is asked about the diagram.

Demo Link

[AIFirst- Syncup-20260508_112243-Meeting Recording.mp4](#)

Presentation link

[ArchVision_AI_Deck.pptx](#)